INSTITUT INTERNATIONAL DE TECHNOLOGIE — ARSI 2ème Année

RAPPORT DE MINI-PROJET

Implémentation d'un VPN sécurisé avec WireGuard sous Linux

Matière	VPN — Administration Réseaux et Sécurité Informatique
Filière	ARSI — 2ème Année
Réalisé par	TOUMI Karim / KALLEL Halim
Encadrant	MASSAOUDI Aymen
Date	Avril 2026

Année Universitaire 2025 — 2026

— Table des Matières

I. Introduction
II. Présentation de WireGuard
III. Environnement de travail
IV. Architecture réseau
V. Configuration réseau des machines virtuelles
VI. Installation de WireGuard
VII. Génération des clés cryptographiques
VIII. Configuration du tunnel VPN
IX. Activation du routage IP et pare-feu
X. Lancement et activation automatique du VPN
XI. Tests de fonctionnement
XII. Analyse du trafic avec Wireshark
XIII. Résultats et discussion
XIV. Conclusion

I. — Introduction

Dans le cadre de notre module d'Administration des Réseaux et de Sécurité Informatique (ARSI), nous avons été amenés à réaliser un mini-projet portant sur la mise en place d'un réseau privé virtuel (VPN). Ce travail s'inscrit dans une démarche pédagogique visant à maîtriser concrètement les mécanismes de sécurisation des communications réseau.

Un réseau privé virtuel (VPN) est une technologie permettant d'établir un tunnel de communication chiffré entre deux ou plusieurs points d'un réseau, garantissant ainsi la confidentialité, l'intégrité et l'authenticité des données transmises. Les VPN sont largement utilisés aussi bien dans les entreprises que dans les environnements personnels pour sécuriser les échanges d'informations.

Objectifs du projet

- Mettre en place un tunnel VPN fonctionnel et sécurisé entre deux machines Linux
- Comprendre le fonctionnement du protocole WireGuard
- Analyser et confirmer le chiffrement du trafic réseau à l'aide de Wireshark
- Documenter l'ensemble de la démarche technique de façon rigoureuse

II. — Présentation de WireGuard

WireGuard est un protocole VPN open-source de nouvelle génération, conçu pour être plus simple, plus performant et plus sûr que les solutions existantes telles qu'OpenVPN ou IPSec. Il a été officiellement intégré dans le noyau Linux à partir de la version 5.6, publiée en mars 2020.

Grâce à son code source particulièrement compact (environ 4 000 lignes contre plus de 100 000 pour OpenVPN), WireGuard offre une surface d'attaque minimale, ce qui facilite les audits de sécurité et réduit le risque de vulnérabilités.

Caractéristiques techniques principales

- Cryptographie moderne : Curve25519 (échange de clés), ChaCha20-Poly1305 (chiffrement), BLAKE2 (hachage)
- Transport basé sur UDP — performances supérieures à TCP pour les tunnels VPN
- Configuration minimaliste et lisible basée sur des paires de clés publique/privée
- Support multi-plateforme : Linux, Windows, macOS, Android, iOS
- Intégration native au noyau Linux depuis la version 5.6 (mars 2020)

Comparatif WireGuard / OpenVPN

Critère	WireGuard	OpenVPN
Vitesse de connexion	Très rapide (noyau)	Modérée (espace utilisateur)
Complexité de config.	Faible	Élevée
Protocole transport	UDP uniquement	TCP ou UDP
Lignes de code	~4 000	~100 000
Intégration noyau	Oui (Linux ≥ 5.6)	Non
Algorithme chiffrement	ChaCha20-Poly1305	AES-256-GCM / CBC

III. — Environnement de travail

L'ensemble du projet a été réalisé dans un environnement entièrement virtualisé grâce à Oracle VirtualBox. Cette approche nous a permis de simuler un réseau complet comprenant un serveur et un client, sans nécessiter de matériel physique supplémentaire.

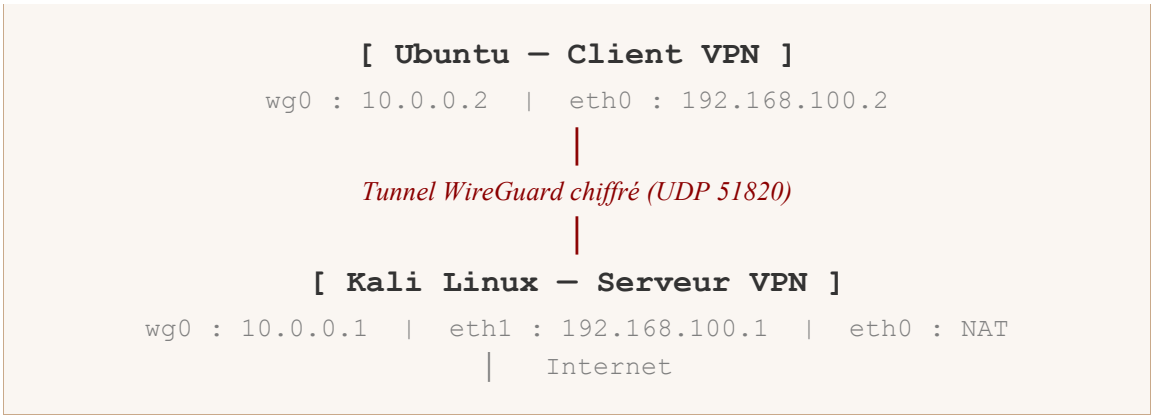
Le choix de Kali Linux comme serveur est pertinent dans le cadre d'un cours de sécurité informatique car, bien qu'il s'agisse d'une distribution orientée audit et test d'intrusion, il est entièrement compatible avec WireGuard et met en évidence la flexibilité du protocole entre différentes distributions Linux.

Hyperviseur	Oracle VirtualBox
Machine serveur	Kali Linux (rolling release)
Machine cliente	Ubuntu Linux (LTS)
Protocole VPN	WireGuard
Port d'écoute	UDP 51820
Réseau virtuel	vpn-net — Internal Network VirtualBox

IV. — Architecture réseau

L'architecture déployée repose sur deux niveaux de réseau superposés : un réseau physique interne VirtualBox reliant les deux machines virtuelles, et un réseau virtuel chiffré WireGuard (overlay) établi au-dessus du précédent.

Schéma de l'architecture



Plan d'adressage IP

Interface	Machine	Adresse IP	Rôle
eth1 / enp0s3	Kali Linux	192.168.100.1	Réseau interne VirtualBox
eth0 / enp0s3	Ubuntu	192.168.100.2	Réseau interne VirtualBox
wg0 (tunnel)	Kali Linux	10.0.0.1/24	Interface VPN WireGuard
wg0 (tunnel)	Ubuntu	10.0.0.2/24	Interface VPN WireGuard

V. — Configuration réseau des machines virtuelles

Avant toute installation de WireGuard, il est indispensable de configurer correctement les interfaces réseau des deux machines virtuelles. Kali Linux dispose de deux cartes réseau (NAT pour l'accès Internet, réseau interne pour la communication avec Ubuntu), tandis qu'Ubuntu n'utilise qu'une seule carte en réseau interne.

Configuration sur Kali Linux — Serveur

Attribution d'une adresse IP statique à l'interface réseau interne (eth1) :

```
# /etc/network/interfaces
auto eth1
iface eth1 inet static
    address 192.168.100.1
    netmask 255.255.255.0
```

Configuration sur Ubuntu — Client

Sur Ubuntu, la configuration réseau est gérée par Netplan :

```
# /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.100.2/24
```

```
sudo netplan apply
```

VI. — Installation de WireGuard

WireGuard est disponible dans les dépôts officiels de Kali Linux et Ubuntu. La procédure d'installation est identique sur les deux machines et ne nécessite que deux commandes :

```
sudo apt update
sudo apt install wireguard -y
wg --version    # Vérification de l'installation
```

La capture d'écran ci-dessous illustre l'installation réussie de WireGuard sur Kali Linux. On peut observer le téléchargement et l'installation des paquets wireguard et wireguard-tools (version 1.0.20250521-1).

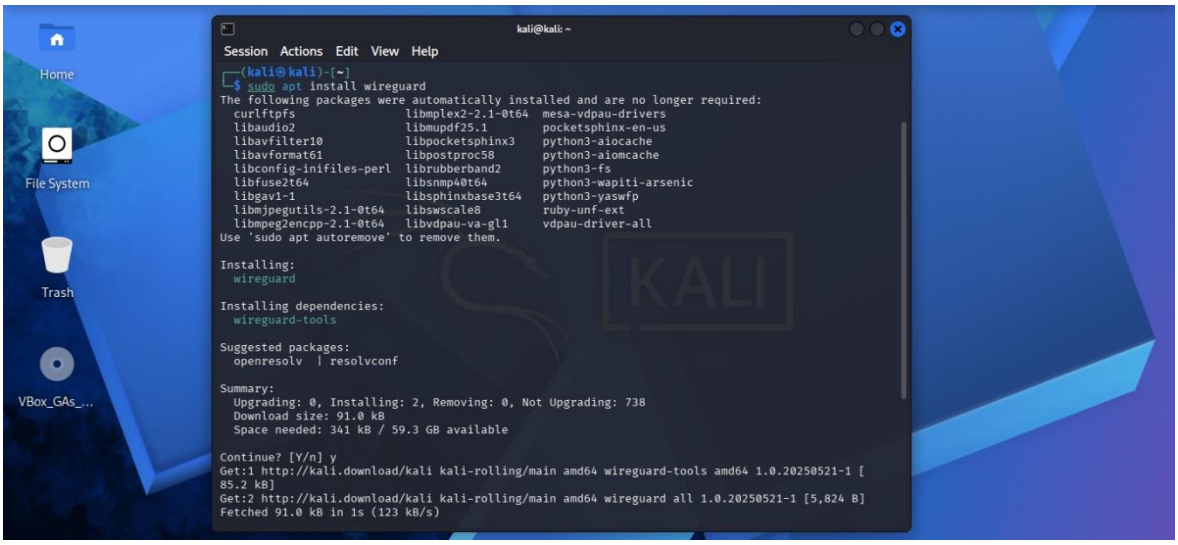


Figure 1 *: Installation de WireGuard sur Kali Linux via apt*

VII. — Génération des clés cryptographiques

WireGuard repose sur un système de cryptographie asymétrique basé sur la courbe elliptique Curve25519. Chaque machine doit générer sa propre paire de clés : une clé privée (confidentielle) et une clé publique (partagée avec le pair). La clé publique est dérivée mathématiquement de la clé privée.

Sur le Serveur — Kali Linux

```
wg genkey | tee server_private.key | wg pubkey > server_public.key
cat server_private.key    # Clé privée du serveur — NE PAS PARTAGER
cat server_public.key     # Clé publique — à communiquer au client
```

Sur le Client — Ubuntu

```
wg genkey | tee client_private.key | wg pubkey > client_public.key
cat client_private.key    # Clé privée du client — NE PAS PARTAGER
cat client_public.key     # Clé publique — à communiquer au serveur
```

Note de sécurité : la clé privée ne doit en aucun cas être transmise ou partagée. Elle doit rester exclusivement sur la machine qui l'a générée. Seule la clé publique est échangée entre les deux machines pour établir le tunnel.

VIII. — Configuration du tunnel VPN

Fichier de configuration — Serveur (Kali Linux)

Création du fichier /etc/wireguard/wg0.conf sur le serveur :

```
[Interface]
Address     = 10.0.0.1/24
ListenPort  = 51820
PrivateKey  = <clé_privée_du_serveur>

[Peer]
PublicKey   = <clé_publique_du_client>
AllowedIPs  = 10.0.0.2/32
```

Fichier de configuration — Client (Ubuntu)

Création du fichier /etc/wireguard/wg0.conf sur le client :

```
[Interface]
Address              = 10.0.0.2/24
PrivateKey           = <clé_privée_du_client>

[Peer]
PublicKey            = <clé_publique_du_serveur>
Endpoint             = 192.168.100.1:51820
AllowedIPs           = 10.0.0.0/24
PersistentKeepalive  = 25
```

IX. — Activation du routage IP et pare-feu

Activation du transfert IPv4 — Kali Linux

Pour permettre à Kali de router les paquets entre le tunnel VPN et le réseau extérieur :

```
sudo nano /etc/sysctl.conf
# Décommenter la ligne suivante :
net.ipv4.ip_forward=1

sudo sysctl -p    # Appliquer sans redémarrage
```

Règle NAT avec iptables

Mise en place du masquerading NAT pour que le client puisse accéder à Internet via le serveur :

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
# Remarque : remplacer eth0 par le nom réel de l'interface NAT (ip a)
```

X. — Lancement et activation automatique du VPN

Démarrage du tunnel WireGuard

La commande suivante est exécutée sur les deux machines pour activer l'interface VPN :

```
sudo wg-quick up wg0      # Sur Kali ET sur Ubuntu
sudo wg                   # Vérification de l'état du tunnel
```

La capture ci-dessous montre l'exécution de la commande wg-quick up wg0 sur Kali Linux. On peut observer les opérations effectuées automatiquement : création de l'interface wg0, chargement de la configuration via /dev/fd/63, attribution de l'adresse IP 10.0.0.1/24 et configuration du MTU à 1420 octets.



Figure 2 : *Lancement du VPN WireGuard avec wg-quick up wg0 sur Kali Linux*

Activation automatique au démarrage

```
sudo systemctl enable wg-quick@wg0
# Le tunnel démarrera automatiquement à chaque démarrage du système
```

XI. — Tests de fonctionnement

Après activation du tunnel VPN sur les deux machines, plusieurs tests ont été réalisés afin de valider le bon fonctionnement de la connexion et de confirmer l'établissement du tunnel WireGuard.

Test 1 — Ping depuis Ubuntu vers Kali

Depuis le client Ubuntu, envoi de paquets ICMP vers l'adresse IP interne du serveur Kali :

```
ping 192.168.100.1
```

```
halim@storage:~/Desktop$ ping 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=0.669 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=2.25 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=0.842 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=0.344 ms
64 bytes from 192.168.100.1: icmp_seq=5 ttl=64 time=1.01 ms
64 bytes from 192.168.100.1: icmp_seq=6 ttl=64 time=0.469 ms
64 bytes from 192.168.100.1: icmp_seq=7 ttl=64 time=0.388 ms
^C
--- 192.168.100.1 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6073ms
rtt min/avg/max/mdev = 0.344/0.853/2.250/0.613 ms
halim@storage:~/Desktop$
```

Figure 3 : *Ping depuis Ubuntu (192.168.100.1) : 7 paquets transmis, 0% de perte, TTL=64*

Test 2 — Ping depuis Kali vers Ubuntu

Depuis le serveur Kali, envoi de paquets ICMP vers l'adresse IP interne du client Ubuntu :

```
ping 192.168.100.2
```

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.100.2
PING 192.168.100.2 (192.168.100.2) 56(84) bytes of data.
64 bytes from 192.168.100.2: icmp_seq=1 ttl=64 time=0.976 ms
64 bytes from 192.168.100.2: icmp_seq=2 ttl=64 time=1.19 ms
64 bytes from 192.168.100.2: icmp_seq=3 ttl=64 time=0.512 ms
64 bytes from 192.168.100.2: icmp_seq=4 ttl=64 time=0.444 ms
64 bytes from 192.168.100.2: icmp_seq=5 ttl=64 time=0.356 ms
^C
── 192.168.100.2 ping statistics ──
5 packets transmitted, 5 received, 0% packet loss, time 4052ms
rtt min/avg/max/mdev = 0.356/0.694/1.185/0.325 ms

┌──(kali㉿kali)-[~]
└─$
```

Figure 4 *: Ping depuis Kali (192.168.100.2) : 5 paquets transmis, 0% de perte, TTL=64*

XII. — Analyse du trafic avec Wireshark

Pour confirmer que WireGuard chiffre effectivement les données en transit, nous avons procédé à une analyse du trafic réseau à l'aide de Wireshark, un analyseur de protocoles réseau open-source.

Installation de Wireshark

```
sudo apt install wireshark -y
sudo wireshark
```

La capture ci-dessous montre l'installation de Wireshark sur Kali Linux ainsi que son interface au démarrage. On peut notamment observer que l'interface wg0 (tunnel VPN) est bien détectée dans la liste des interfaces réseau disponibles, confirmant que le tunnel est actif.

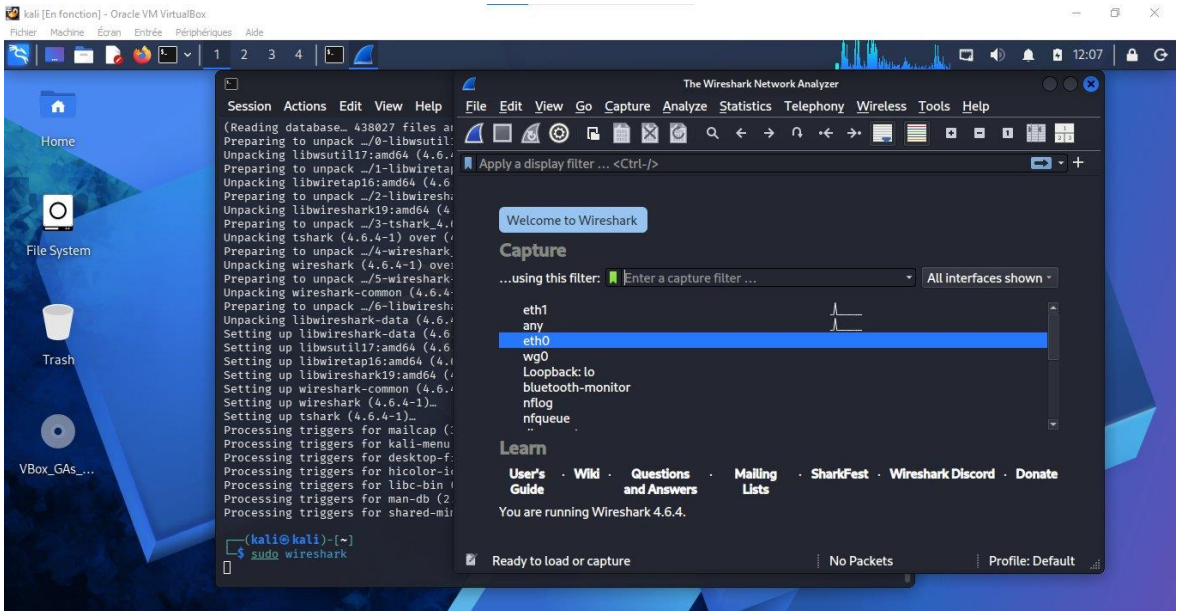


***Figure 5 :**: Installation de Wireshark et détection de l'interface wg0 dans la liste des interfaces*

Capture et lancement

Démarrage d'une capture live sur l'interface eth0 de Kali pendant l'activité réseau entre les deux VMs :

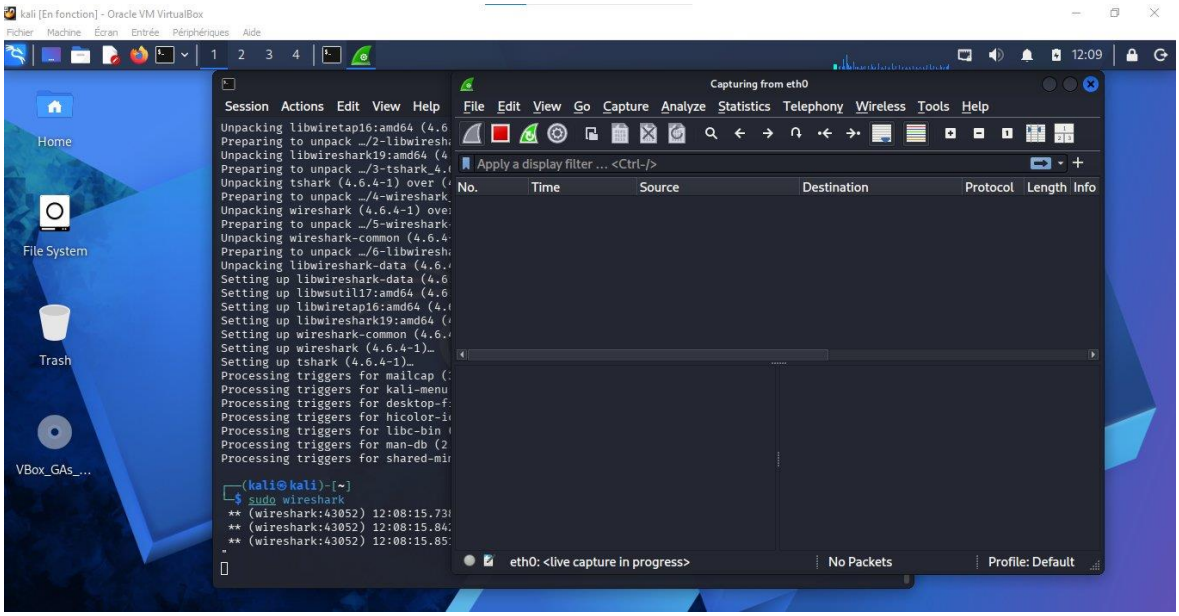


Figure 6 *: Wireshark 4.6.4 en cours de capture sur eth0 — Kali Linux*

Analyse des paquets WireGuard

Application du filtre d'affichage pour isoler le trafic VPN WireGuard :

```
udp.port == 51820
```

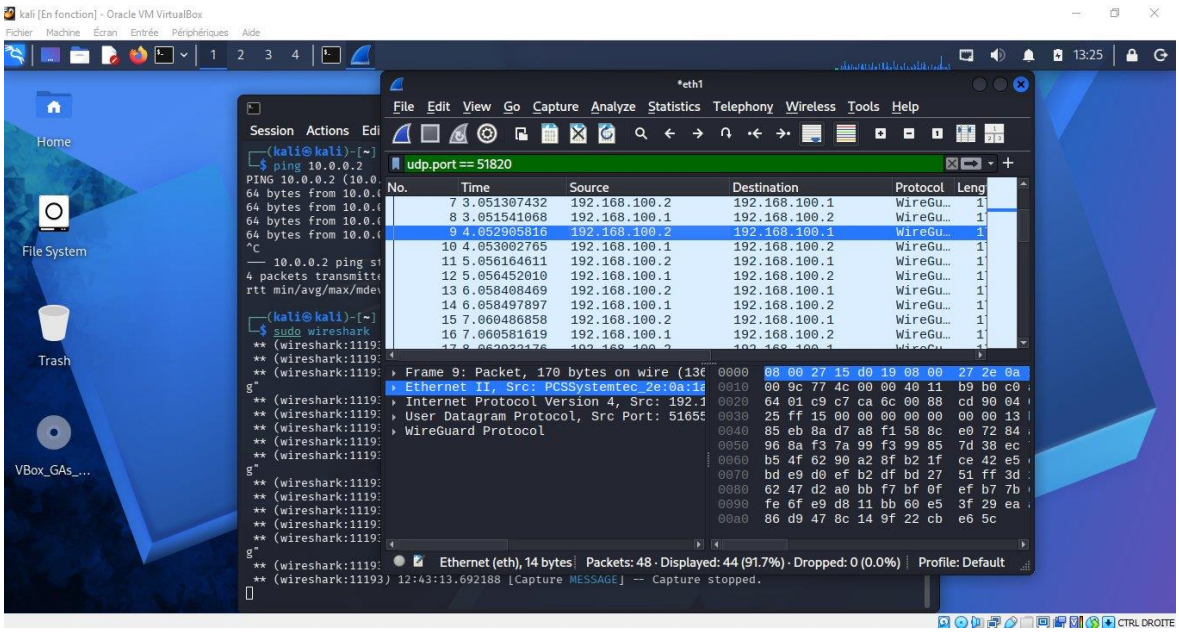


Figure 7 *: Filtre udp.port==51820 : trafic WireGuard chiffré entre 192.168.100.2 et 192.168.100.1*

Observations et analyse

- Le trafic VPN WireGuard utilise exclusivement le protocole UDP sur le port 51820
- 44 paquets WireGuard sont visibles sur un total de 48 capturés, soit 91,7% du trafic
- La source et la destination des paquets correspondent aux adresses IP physiques (192.168.100.2 et 192.168.100.1)
- Le contenu des paquets est totalement chiffré par ChaCha20-Poly1305 : aucune donnée lisible n'est accessible
- WireGuard Protocol est identifié par Wireshark mais le payload reste opaque, preuve du chiffrement effectif

XIII. — Résultats et discussion

L'ensemble des objectifs définis dans le cahier des charges a été atteint avec succès. Le tableau ci-dessous présente un bilan récapitulatif des livrables et de leur état de réalisation.

Objectif / Livrable	Statut	Référence
Tunnel VPN établi et fonctionnel	✓ **Réalisé**	`wg-quick up wg0`
Ping Ubuntu → Kali (0% perte)	✓ **Réalisé**	`Figure 3`
Ping Kali → Ubuntu (0% perte)	✓ **Réalisé**	`Figure 4`
Chiffrement vérifié par Wireshark	✓ **Confirmé**	`Figure 7`
Démarrage automatique configuré	✓ **Réalisé**	`systemctl enable`

Points forts observés

- Simplicité de déploiement : configuration réduite à deux fichiers courts et lisibles
- Performances élevées : latence inférieure à 2 ms dans l'environnement VirtualBox
- Sécurité confirmée : trafic totalement chiffré et illisible dans Wireshark
- Interopérabilité : fonctionne parfaitement entre Kali Linux et Ubuntu

XIV. — Conclusion

Ce mini-projet nous a permis de mettre en pratique de manière concrète et exhaustive les concepts fondamentaux des réseaux privés virtuels, en déployant une solution complète avec WireGuard entre deux machines Linux virtualisées. Nous avons couvert l'intégralité du cycle de vie d'un déploiement VPN : planification de l'architecture réseau, configuration des machines, installation, génération des clés cryptographiques, configuration du tunnel, tests de connectivité et analyse forensique du trafic.

WireGuard s'est révélé être un protocole remarquablement efficace et accessible. Sa conception minimaliste, ses algorithmes cryptographiques modernes et ses performances natives au noyau Linux en font un choix de premier plan pour sécuriser les communications réseau, aussi bien dans des contextes pédagogiques que professionnels. L'analyse Wireshark a validé de manière irréfutable le chiffrement intégral du trafic transmis dans le tunnel.

Ce projet a également renforcé nos compétences en administration Linux, en virtualisation avec VirtualBox et en analyse de protocoles réseau — autant de savoir-faire directement applicables en situation professionnelle.

Perspectives d'amélioration

- Déploiement avec plusieurs clients VPN connectés simultanément
- Migration vers un environnement cloud public (AWS, Azure ou OVH)
- Automatisation complète de la configuration via des scripts Bash ou Ansible
- Intégration d'une politique de sécurité renforcée avec pare-feu avancé (nftables)